

Mnbara دليل التوسع الشامل لمنصة

Plugins نظام + CrafterCMS + eBay Live إضافة

المحتويات

1. [إضافة خدمة eBay Live](#)
2. [دمج CrafterCMS](#)
3. [القابل للتوسع Plugins نظام](#)
4. [البنية الشاملة النهائية](#)
5. [خطة التنفيذ](#)

1 eBay Live إضافة خدمة {#1-ebay-live-service}

ما هو eBay Live؟

هو منصة تسوق تفاعلية في الوقت الفعلي تتيح للمشتريين شراء المنتجات خلال البث المباشر، مع القدرة على التفاعل مع البائع والمشاركة في eBay Live. مزادات حية.

المكونات الرئيسية



ebay-live-service/	
— streaming/	# إدارة البث المباشر
— rtmp-server/	# RTMP streaming server
— hls-converter/	# للمتصفحات HLS تحويل
— webrtc-gateway/	# للتفاعل WebRTC
— auction/	# نظام المزادات الحية
— bidding-engine/	# محرك المزايدة
— soft-close/	# نظام الإغلاق التدريجي
— payment-capture/	# التقاط الدفع التلقائي
— chat/	# نظام الدردشة الحية
— websocket-server/	# للدردشة WebSocket
— moderation/	# إدارة المحتوى
— emoji-reactions/	# ردود الأفعال
— products/	# إدارة المنتجات في البث
— carousel/	# عرض المنتجات
— pin-product/	# تثبيت المنتج
— inventory-sync/	# مزامنة المخزون
— analytics/	# تحليلات البث
— viewer-tracking/	# تتبع المشاهدين
— engagement/	# معدلات التفاعل
— sales-metrics/	# مقاييس المبيعات

الوقت المتوقع للتطوير

1 المرحلة: MVP (4-6 أسابيع)



yaml

الأسبوع 1-2: البنية الأساسية

- إعداد RTMP server
- HLS basic تحويل
- WebSocket للردشة الأساسية
- UI للبت المباشر (React)

الأسبوع 3-4: الوظائف الأساسية

- نظام المزادات البسيط
- عرض المنتجات (carousel)
- الدفع التلقائي للفائزين
- إشعارات الوقت الفعلي

الأسبوع 5-6: الاختبار والتحسين

- اختبار الأحمال
- تحسين الكمون (latency)
- UI/UX polishing
- Integration testing

2 (أسابيع 3-4) Enhanced Features: المرحلة



yaml

الميزات المتقدمة:

- Soft-close auctions (الإغلاق التدريجي)
- Multi-camera switching
- Analytics dashboard
- Advanced moderation
- Video receipts
- Replay functionality

البنية التقنية

Backend Stack



javascript

```
// streaming-service/src/index.js
const express = require('express');
const NodeMediaServer = require('node-media-server');
const socket = require('socket.io');

// RTMP Server Configuration
const rtmpConfig = {
  rtmp: {
    port: 1935,
    chunk_size: 60000,
    gop_cache: true,
    ping: 30,
    ping_timeout: 60
  },
  http: {
    port: 8000,
    allow_origin: '*',
    mediaroot: './media'
  },
  trans: {
    ffmpeg: '/usr/bin/ffmpeg',
    tasks: [{
      app: 'live',
      hls: true,
      hlsFlags: '[hls_time=2:hls_list_size=3:hls_flags=delete_segments]',
      dash: true,
      dashFlags: '[f=dash:window_size=3:extra_window_size=5]'
    }]
  }
};
```

```
class LiveStreamingService {
  constructor() {
    this.nms = new NodeMediaServer(rtmpConfig);
    this.io = null;
    this.activeStreams = new Map();
  }

  async start() {
    // Start RTMP server
    this.nms.run();

    // Setup WebSocket for chat/bidding
    this.setupWebSocket();

    // Stream events
    this.nms.on('prePublish', this.handleStreamStart.bind(this));
    this.nms.on('donePublish', this.handleStreamEnd.bind(this));
  }

  setupWebSocket() {
    this.io = socket(8080, {
      cors: { origin: '*' }
    });

    this.io.on('connection', (socket) => {
      console.log('User connected:', socket.id);

      socket.on('join-stream', (streamId) => {
```

```

    socket.join(`stream-${streamId}`);
    this.updateViewerCount(streamId);
  });

  socket.on('send-message', (data) => {
    this.io.to(`stream-${data.streamId}`).emit('new-message', data);
  });

  socket.on('place-bid', async (bidData) => {
    await this.processBid(bidData);
  });
}

async handleStreamStart(id, streamPath, args) {
  const streamId = this.extractStreamId(streamPath);

  this.activeStreams.set(streamId, {
    startTime: Date.now(),
    viewers: 0,
    status: 'live'
  });

  // Notify all services
  await this.notifyStreamStart(streamId);
}

async processBid(bidData) {
  // Validate bid
  const isValid = await this.validateBid(bidData);

  if (!isValid) {
    return { success: false, error: 'Invalid bid' };
  }

  // Check if highest bid
  const currentHighest = await this.getCurrentHighestBid(bidData.auctionId);

  if (bidData.amount <= currentHighest) {
    return { success: false, error: 'Bid too low' };
  }

  // Process bid
  await this.saveBid(bidData);

  // Broadcast to all viewers
  this.io.to(`stream-${bidData.streamId}`).emit('new-bid', {
    auctionId: bidData.auctionId,
    amount: bidData.amount,
    bidder: bidData.userId,
    timestamp: Date.now()
  });

  // Check for soft-close extension
  await this.checkSoftClose(bidData.auctionId);

  return { success: true };
}

```

```
async checkSoftClose(auctionId) {
  const auction = await this.getAuction(auctionId);
  const timeRemaining = auction.endTime - Date.now();

  // If less than 5 seconds remaining, extend by 5 seconds
  if (timeRemaining < 5000) {
    auction.endTime += 5000;
    await this.updateAuction(auction);

    this.io.to(`stream-${auction.streamId}`).emit('auction-extended', {
      auctionId,
      newEndTime: auction.endTime
    });
  }
}
}
```

module.exports = LiveStreamingService;

Frontend Component (React)



typescript

```
// components/LiveStream/LiveStreamViewer.tsx
import React, { useEffect, useState, useRef } from 'react';
import Hls from 'hls.js';
import { io, Socket } from 'socket.io-client';

interface LiveStreamViewerProps {
  streamId: string;
  userId: string;
}

export const LiveStreamViewer: React.FC<LiveStreamViewerProps> = ({
  streamId,
  userId
}) => {
  const videoRef = useRef<HTMLVideoElement>(null);
  const [socket, setSocket] = useState<Socket | null>(null);
  const [messages, setMessages] = useState<any[]>([]);
  const [currentBid, setCurrentBid] = useState<number>(0);
  const [products, setProducts] = useState<any[]>([]);
  const [pinnedProduct, setPinnedProduct] = useState<any>(null);

  useEffect(() => {
    // Initialize HLS player
    if (Hls.isSupported() && videoRef.current) {
      const hls = new Hls();
      hls.loadSource(`http://localhost:8000/live/${streamId}/index.m3u8`);
      hls.attachMedia(videoRef.current);
    }

    // Connect to WebSocket
    const newSocket = io('http://localhost:8080');
    setSocket(newSocket);

    newSocket.emit('join-stream', streamId);

    // Listen for messages
    newSocket.on('new-message', (msg) => {
      setMessages(prev => [...prev, msg]);
    });

    // Listen for bids
    newSocket.on('new-bid', (bid) => {
      setCurrentBid(bid.amount);
    });

    // Listen for product updates
    newSocket.on('product-pinned', (product) => {
      setPinnedProduct(product);
    });

    return () => {
      newSocket.disconnect();
    };
  }, [streamId]);

  const placeBid = async (amount: number) => {
    if (!socket) return;

    socket.emit('place-bid', {

```

```
    streamId,  
    auctionId: pinnedProduct?.auctionId,  
    userId,  
    amount  
  });  
};
```

```
const sendMessage = (text: string) => {  
  if (!socket) return;
```

```
  socket.emit('send-message', {  
    streamId,  
    userId,  
    text,  
    timestamp: Date.now()  
  });  
};
```

```
return (  
  <div className="live-stream-container">  
    <div className="video-section">  
      <video  
        ref={videoRef}  
        controls  
        autoPlay  
        className="live-video"  
      />  
  
      { /* Product Carousel */ }  
      <div className="product-carousel">  
        {products.map(product => (  
          <ProductCard  
            key={product.id}  
            product={product}  
            isPinned={pinnedProduct?.id === product.id}  
          />  
        ))}  
      </div>  
    </div>  
  
    <div className="interaction-section">  
      { /* Chat */ }  
      <ChatBox  
        messages={messages}  
        onSendMessage={sendMessage}  
      />  
  
      { /* Bidding Panel */ }  
      {pinnedProduct && (  
        <BiddingPanel  
          product={pinnedProduct}  
          currentBid={currentBid}  
          onPlaceBid={placeBid}  
        />  
      )}  
    </div>  
  </div>  
);  
};
```

قاعدة البيانات



sql

-- Streaming Events Table

```
CREATE TABLE live_streams (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  seller_id UUID NOT NULL REFERENCES users(id),  
  title VARCHAR(255) NOT NULL,  
  description TEXT,  
  category VARCHAR(100),  
  scheduled_start TIMESTAMP NOT NULL,  
  actual_start TIMESTAMP,  
  end_time TIMESTAMP,  
  status VARCHAR(20) DEFAULT 'scheduled', -- scheduled, live, ended  
  stream_key VARCHAR(255) UNIQUE,  
  rtmp_url TEXT,  
  hls_url TEXT,  
  total_viewers INT DEFAULT 0,  
  peak_viewers INT DEFAULT 0,  
  total_sales DECIMAL(10,2) DEFAULT 0,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Live Auctions Table

```
CREATE TABLE live_auctions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  stream_id UUID NOT NULL REFERENCES live_streams(id),  
  product_id UUID NOT NULL REFERENCES products(id),  
  starting_bid DECIMAL(10,2) NOT NULL,  
  current_bid DECIMAL(10,2) NOT NULL,  
  highest_bidder_id UUID REFERENCES users(id),  
  start_time TIMESTAMP NOT NULL,  
  end_time TIMESTAMP NOT NULL,  
  soft_close_enabled BOOLEAN DEFAULT TRUE,  
  status VARCHAR(20) DEFAULT 'active', -- active, sold, unsold  
  winner_id UUID REFERENCES users(id),  
  final_price DECIMAL(10,2),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Bids Table

```
CREATE TABLE live_bids (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  auction_id UUID NOT NULL REFERENCES live_auctions(id),  
  bidder_id UUID NOT NULL REFERENCES users(id),  
  amount DECIMAL(10,2) NOT NULL,  
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  is_winning BOOLEAN DEFAULT FALSE  
);
```

-- Chat Messages Table

```
CREATE TABLE stream_messages (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  stream_id UUID NOT NULL REFERENCES live_streams(id),  
  user_id UUID NOT NULL REFERENCES users(id),  
  message TEXT NOT NULL,  
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  is_deleted BOOLEAN DEFAULT FALSE  
);
```

-- Stream Analytics Table

```
CREATE TABLE stream_analytics (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  stream_id UUID NOT NULL REFERENCES live_streams(id),  
  metric_type VARCHAR(50), -- viewers, engagement, sales  
  metric_value JSONB,  
  recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

🔧 Integration مع المشروع الحالي



typescript

```
// services/ebay-live-service/src/integration/index.ts
import { OrderService } from '@mnbara/order-service';
import { PaymentService } from '@mnbara/payment-service';
import { NotificationService } from '@mnbara/notification-service';
import { InventoryService } from '@mnbara/inventory-service';

export class LiveStreamIntegration {
  constructor(
    private orderService: OrderService,
    private paymentService: PaymentService,
    private notificationService: NotificationService,
    private inventoryService: InventoryService
  ) {}

  async handleAuctionWin(auctionData: AuctionWinData) {
    // 1. Create order automatically
    const order = await this.orderService.createOrder({
      userId: auctionData.winnerId,
      productId: auctionData.productId,
      price: auctionData.finalPrice,
      type: 'live_auction',
      streamId: auctionData.streamId
    });

    // 2. Process payment (auto-charged)
    const payment = await this.paymentService.processPayment({
      orderId: order.id,
      userId: auctionData.winnerId,
      amount: auctionData.finalPrice,
      paymentMethodId: auctionData.paymentMethodId
    });

    // 3. Update inventory
    await this.inventoryService.reduceStock({
      productId: auctionData.productId,
      quantity: 1
    });

    // 4. Send notifications
    await this.notificationService.send({
      userId: auctionData.winnerId,
      type: 'auction_win',
      data: {
        productName: auctionData.productName,
        finalPrice: auctionData.finalPrice,
        orderId: order.id
      }
    });

    // 5. Create video receipt
    await this.createVideoReceipt({
      streamId: auctionData.streamId,
      orderId: order.id,
      timestamp: auctionData.winTimestamp
    });

    return { order, payment };
  }
}
```

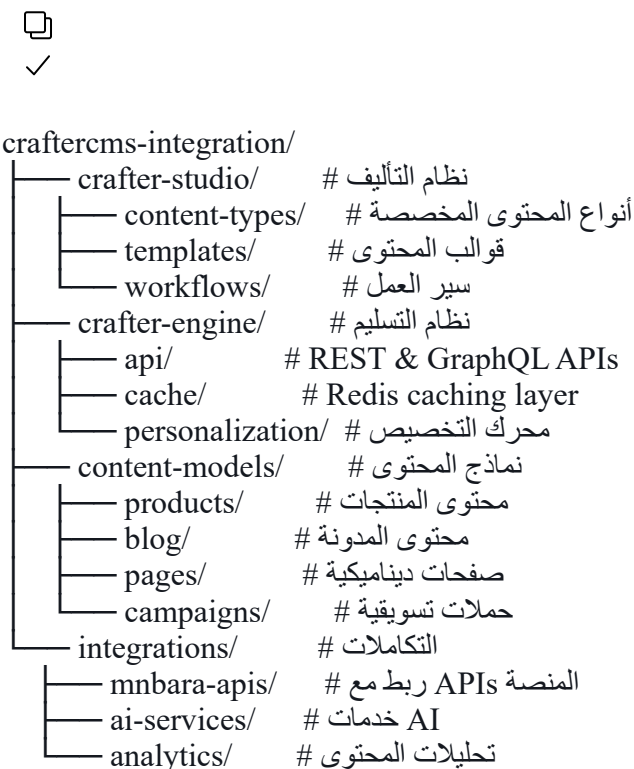
```
async createVideoReceipt(data: VideoReceiptData) {  
  // Extract video segment at specific timestamp  
  // Store as replay link for user  
  // Available for 180 days  
}
```

2 دمج CrafterCMS {#2-craftercms-integration}

ما هو CrafterCMS؟

قابلة للتوسع MACH أخرى، مع بنية frontend وأي تقنية React يدعم، وGit وJava مبني على headless هو نظام إدارة محتوى CrafterCMS.

البنية المعمارية



الوقت المتوقع للتطوير

المرحلة 1: الإعداد الأساسي (2-3 أسابيع)

yaml

الأسبوع 1:

- ☒ تثبيت CrafterCMS (Docker/K8s)
- ☒ للمحتوى Git repository إعداد
- ☒ أساسية content types إنشاء
- ☒ ربط مع قاعدة البيانات الحالية

الأسبوع 2-3:

- ☒ إنشاء نماذج محتوى للمنتجات
- ☒ إعداد GraphQL APIs
- ☒ frontend (React) دمج مع
- ☒ preview environment إعداد

المرحلة 2: التخصيص والتكامل (3-4 أسابيع)



yaml

الميزات المتقدمة:

- (personalization) نظام التخصيص
- Multilingual content
- A/B testing للمحتوى
- AI-powered content suggestions
- Advanced workflows

 التكامل التقني

Content Types Definition



yaml

```
# /config/studio/content-types/product/form-definition.xml
<?xml version="1.0" encoding="UTF-8"?>
<form>
  <title>Product Content</title>
  <description>Enhanced product content with rich media</description>

  <sections>
    <section>
      <title>Basic Information</title>
      <fields>
        <field>
          <type>input</type>
          <id>productId</id>
          <title>Product ID (from DB)</title>
          <readonly>true</readonly>
        </field>

        <field>
          <type>wysiwyg</type>
          <id>description</id>
          <title>Rich Description</title>
          <features>
            <feature>bold</feature>
            <feature>italic</feature>
            <feature>link</feature>
            <feature>image</feature>
            <feature>video</feature>
          </features>
        </field>

        <field>
          <type>image</type>
          <id>heroImage</id>
          <title>Hero Image</title>
          <imageManager>cloudinary</imageManager>
        </field>

        <field>
          <type>repeat</type>
          <id>gallery</id>
          <title>Product Gallery</title>
          <fields>
            <field>
              <type>image</type>
              <id>image</id>
            </field>
            <field>
              <type>input</type>
              <id>caption</id>
            </field>
          </fields>
        </field>
      </fields>
    </section>

    <section>
      <title>Marketing Content</title>
      <fields>
        <field>
```

```
<type>wysiwyg</type>
<id>highlights</id>
<title>Key Highlights</title>
</field>

<field>
  <type>video</type>
  <id>demoVideo</id>
  <title>Demo Video</title>
  <videoProvider>youtube,vimeo</videoProvider>
</field>

<field>
  <type>component-collection</type>
  <id>testimonials</id>
  <title>Customer Testimonials</title>
</field>
</fields>
</section>

<section>
  <title>SEO & Metadata</title>
  <fields>
    <field>
      <type>input</type>
      <id>metaTitle</id>
      <title>Meta Title</title>
    </field>

    <field>
      <type>textarea</type>
      <id>metaDescription</id>
      <title>Meta Description</title>
    </field>

    <field>
      <type>tag-selector</type>
      <id>keywords</id>
      <title>Keywords</title>
    </field>
  </fields>
</section>
</sections>
</form>
```

GraphQL API Integration



typescript

```
// services/content-service/src/craftercms-client.ts
import { GraphQLClient } from 'graphql-request';
```

```
export class CrafterCMSClient {
  private client: GraphQLClient;
```

```
  constructor() {
    this.client = new GraphQLClient('http://localhost:8080/api/1/site/graphql', {
      headers: {
        'Content-Type': 'application/json'
      }
    });
  }
}
```

```
async getProductContent(productId: string) {
  const query = `
    query GetProductContent($productId: String!) {
      product(productId: $productId) {
        description
        heroImage {
          url
          alt
        }
        gallery {
          image {
            url
          }
          caption
        }
        highlights
        demoVideo {
          url
          provider
          thumbnail
        }
        testimonials {
          customerName
          rating
          comment
          date
        }
        seo {
          metaTitle
          metaDescription
          keywords
        }
      }
    }
  `;
}
```

```
const data = await this.client.request(query, { productId });
return data.product;
}
```

```
async updateProductContent(productId: string, content: any) {
  // Update content via Crafter API
  const response = await fetch(`http://localhost:8080/studio/api/1/services/content/write-content.json`, {
    method: 'POST',
    headers: {
```



```

    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    site: 'mnbara',
    path: `/content/products/${productId}.xml`,
    content: this.convertToXML(content)
  })
});

return response.json();
}

// AI-Powered Content Generation
async generateProductDescription(productData: any) {
  const prompt = `
    Generate compelling product description for:
    Name: ${productData.name}
    Category: ${productData.category}
    Features: ${productData.features.join(', ')}
  `;

  const aiResponse = await this.callAIService(prompt);
  return aiResponse.description;
}

// Content Personalization
async getPersonalizedContent(userId: string, productId: string) {
  const userProfile = await this.getUserProfile(userId);
  const baseContent = await this.getProductContent(productId);

  // Apply personalization rules
  return this.personalizeContent(baseContent, userProfile);
}
}

```

React Integration





 typescript

```
// frontend/src/hooks/useCrafterContent.ts
import { useQuery } from '@tanstack/react-query';
import { crafterClient } from '@services/crafter-client';

export function useCrafterContent(productId: string) {
  return useQuery({
    queryKey: ['crafter-content', productId],
    queryFn: () => crafterClient.getProductContent(productId),
    staleTime: 1000 * 60 * 5, // 5 minutes
  });
}

// Component usage
function ProductPage({ productId }: { productId: string }) {
  const { data: productData } = useProduct(productId); // From your API
  const { data: contentData } = useCrafterContent(productId); // From CrafterCMS

  return (
    <div className="product-page">
      { /* Base product info from your DB */ }
      <h1>{productData.name}</h1>
      <p className="price">${productData.price}</p>

      { /* Rich content from CrafterCMS */ }
      <div
        className="rich-description"
        dangerouslySetInnerHTML={{ __html: contentData.description }}
      />

      <img src={contentData.heroImage.url} alt={contentData.heroImage.alt} />

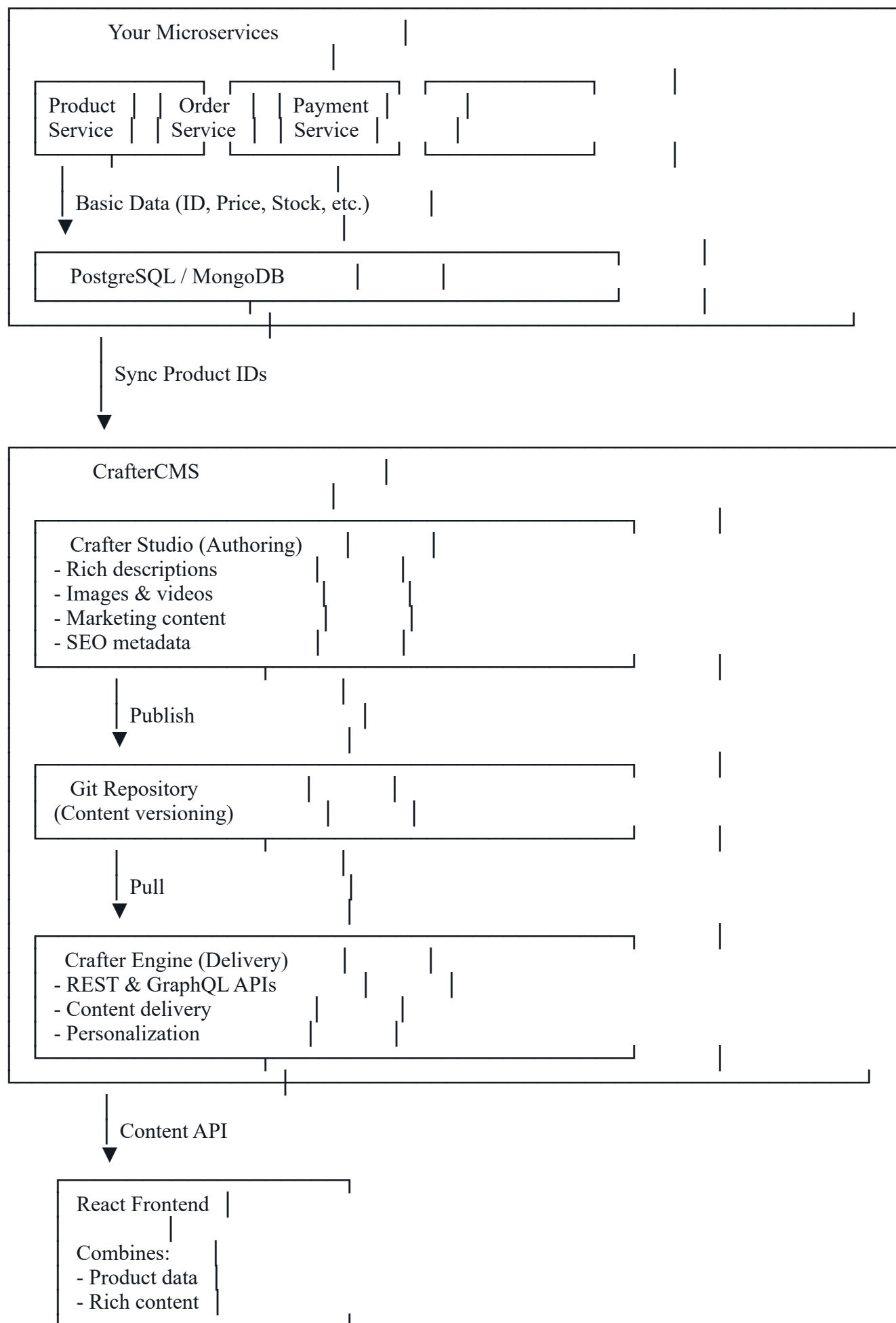
      { /* Product gallery */ }
      <Gallery images={contentData.gallery} />

      { /* Demo video */ }
      {contentData.demoVideo && (
        <VideoPlayer url={contentData.demoVideo.url} />
      )}

      { /* Customer testimonials */ }
      <Testimonials data={contentData.testimonials} />
    </div>
  );
}
```

Workflow مع المشروع الحالي





- البيانات الأساسية → Your microservices
- المحتوى الغني → CrafterCMS

2. تمكين فريق المحتوى

- سهولة WYSIWYG واجهة
- لا يحتاجون للمطورين لكل تغيير
- Preview النشر

3. Personalization

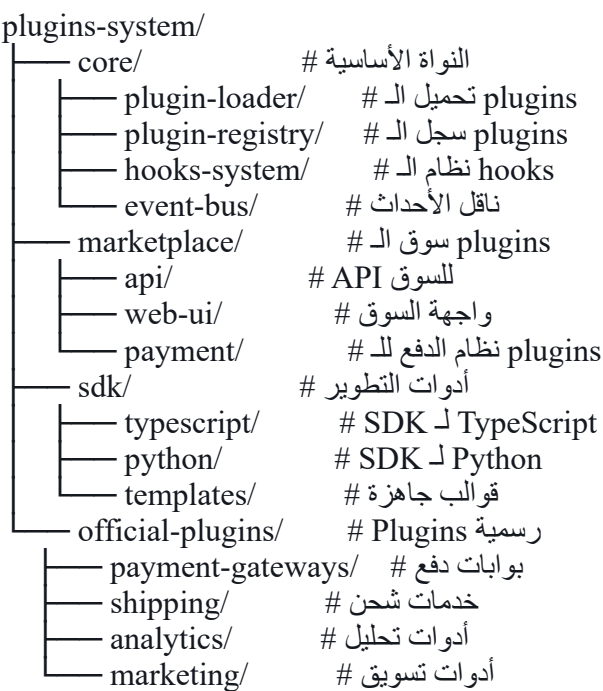
- محتوى مخصص لكل مستخدم
- A/B testing
- Dynamic content

4. Performance

- Content caching
- CDN integration
- Fast content delivery

3 {#3-plugin-system} القابل للتوسع Plugins نظام

🔌 Plugins البنية المعمارية للـ



🎯 Plugin Architecture

1. Plugin Manifest



json

```
// plugin.json
{
  "name": "@mnbara/stripe-payment-plugin",
  "version": "1.0.0",
  "description": "Stripe payment gateway integration",
  "author": "Mnbara Team",
  "license": "MIT",

  "mnbara": {
    "minVersion": "1.0.0",
    "maxVersion": "2.0.0"
  },

  "type": "payment-gateway",

  "hooks": [
    "payment.process",
    "payment.refund",
    "payment.webhook"
  ],

  "permissions": [
    "orders.read",
    "orders.update",
    "webhooks.create"
  ],

  "config": {
    "apiKey": {
      "type": "string",
      "required": true,
      "secret": true
    },
    "webhookSecret": {
      "type": "string",
      "required": true,
      "secret": true
    }
  },

  "dependencies": {
    "stripe": "^12.0.0"
  },

  "entry": "dist/index.js"
}
```

2. Plugin SDK



typescript

```

// @mnbara/plugin-sdk/src/index.ts
export abstract class MnbaraPlugin {
  protected app: MnbaraApp;
  protected config: any;
  protected logger: Logger;

  constructor(app: MnbaraApp, config: any) {
    this.app = app;
    this.config = config;
    this.logger = app.createLogger(this.getName());
  }

  // Required methods
  abstract getName(): string;
  abstract getVersion(): string;
  abstract initialize(): Promise<void>;
  abstract destroy(): Promise<void>;

  // Hook registration
  protected registerHook(hookName: string, handler: Function) {
    this.app.hooks.register(hookName, handler);
  }

  // Event emission
  protected emit(eventName: string, data: any) {
    this.app.eventBus.emit(eventName, data);
  }

  // Service access
  protected getService<T>(serviceName: string): T {
    return this.app.services.get(serviceName);
  }

  // Database access (with permissions)
  protected async query(sql: string, params?: any[]) {
    if (!this.hasPermission('database.query')) {
      throw new Error('Permission denied');
    }
    return this.app.database.query(sql, params);
  }

  // API registration
  protected registerAPI(routes: PluginRoute[]) {
    routes.forEach(route => {
      this.app.api.registerRoute({
        ...route,
        plugin: this.getName()
      });
    });
  }

  // Example: Payment Gateway Plugin Interface
  export abstract class PaymentGatewayPlugin extends MnbaraPlugin {
    abstract processPayment(order: Order, paymentData: PaymentData): Promise<PaymentResult>;
    abstract refundPayment(transactionId: string, amount: number): Promise<RefundResult>;
    abstract handleWebhook(payload: any): Promise<void>;
    abstract getPaymentMethods(): PaymentMethod[];
  }

```

3. Example Plugin Implementation



typescript

```
// plugins/stripe-payment/src/index.ts
import { PaymentGatewayPlugin } from '@mnbara/plugin-sdk';
import Stripe from 'stripe';

export default class StripePaymentPlugin extends PaymentGatewayPlugin {
  private stripe: Stripe;

  getName() {
    return '@mnbara/stripe-payment';
  }

  getVersion() {
    return '1.0.0';
  }

  async initialize() {
    this.stripe = new Stripe(this.config.apiKey, {
      apiVersion: '2023-10-16'
    });

    // Register webhook endpoint
    this.registerAPI([
      {
        method: 'POST',
        path: '/webhooks/stripe',
        handler: this.handleWebhook.bind(this)
      }
    ]);

    // Register payment hooks
    this.registerHook('payment.process', this.processPayment.bind(this));
    this.registerHook('payment.refund', this.refundPayment.bind(this));

    this.logger.info('Stripe plugin initialized');
  }

  async processPayment(order: Order, paymentData: PaymentData) {
    try {
      const paymentIntent = await this.stripe.paymentIntents.create({
        amount: Math.round(order.total * 100), // Convert to cents
        currency: order.currency.toLowerCase(),
        payment_method: paymentData.paymentMethodId,
        confirm: true,
        metadata: {
          orderId: order.id,
          customerId: order.customerId
        }
      });

      // Emit event
      this.emit('payment.completed', {
        orderId: order.id,
        transactionId: paymentIntent.id,
        amount: order.total
      });

      return {
        success: true,
        transactionId: paymentIntent.id,
      };
    }
  }
}
```



```

    status: paymentIntent.status
  };
} catch (error) {
  this.logger.error('Payment failed:', error);

  return {
    success: false,
    error: error.message
  };
}
}

async refundPayment(transactionId: string, amount: number) {
  const refund = await this.stripe.refunds.create({
    payment_intent: transactionId,
    amount: Math.round(amount * 100)
  });

  return {
    success: true,
    refundId: refund.id
  };
}

async handleWebhook(payload: any) {
  const event = this.stripe.webhooks.constructEvent(
    payload.body,
    payload.headers['stripe-signature'],
    this.config.webhookSecret
  );

  switch (event.type) {
    case 'payment_intent.succeeded':
      await this.handlePaymentSuccess(event.data.object);
      break;
    case 'payment_intent.payment_failed':
      await this.handlePaymentFailure(event.data.object);
      break;
  }
}

getPaymentMethods() {
  return [
    {
      id: 'stripe_card',
      name: 'Credit/Debit Card',
      type: 'card',
      icon: '/assets/stripe-card.svg'
    }
  ];
}

async destroy() {
  // Cleanup
  this.logger.info('Stripe plugin destroyed');
}
}

```

4. Plugin Loader System



typescript

```
// core/plugin-loader/src/index.ts
import { promises as fs } from 'fs';
import path from 'path';

export class PluginLoader {
  private pluginsDir: string;
  private loadedPlugins: Map<string, LoadedPlugin> = new Map();
  private registry: PluginRegistry;

  constructor(pluginsDir: string, registry: PluginRegistry) {
    this.pluginsDir = pluginsDir;
    this.registry = registry;
  }

  async loadAllPlugins() {
    const pluginDirs = await fs.readdir(this.pluginsDir);

    for (const dir of pluginDirs) {
      await this.loadPlugin(dir);
    }
  }

  async loadPlugin(pluginName: string) {
    const pluginPath = path.join(this.pluginsDir, pluginName);

    // Read manifest
    const manifestPath = path.join(pluginPath, 'plugin.json');
    const manifest = JSON.parse(await fs.readFile(manifestPath, 'utf-8'));

    // Validate manifest
    this.validateManifest(manifest);

    // Check platform compatibility
    this.checkCompatibility(manifest);

    // Load plugin code
    const entryPath = path.join(pluginPath, manifest.entry);
    const PluginClass = (await import(entryPath)).default;

    // Read config
    const config = await this.loadPluginConfig(pluginName);

    // Instantiate plugin
    const pluginInstance = new PluginClass(this.app, config);

    // Initialize
    await pluginInstance.initialize();

    // Register
    this.loadedPlugins.set(pluginName, {
      manifest,
      instance: pluginInstance,
      loadedAt: new Date()
    });

    this.registry.register(manifest);

    console.log(`Plugin ${pluginName} loaded successfully`);
  }
}
```

```

async unloadPlugin(pluginName: string) {
  const plugin = this.loadedPlugins.get(pluginName);
  if (!plugin) return;

  // Destroy plugin instance
  await plugin.instance.destroy();

  // Unregister hooks
  this.app.hooks.unregisterAll(pluginName);

  // Unregister routes
  this.app.api.unregisterRoutes(pluginName);

  // Remove from loaded plugins
  this.loadedPlugins.delete(pluginName);

  this.registry.unregister(pluginName);

  console.log(`Plugin ${pluginName} unloaded`);
}

async reloadPlugin(pluginName: string) {
  await this.unloadPlugin(pluginName);
  await this.loadPlugin(pluginName);
}

getLoadedPlugins() {
  return Array.from(this.loadedPlugins.values());
}
}

```

5. Plugin Marketplace API



typescript

```
// marketplace/api/src/index.ts
export class PluginMarketplaceAPI {

  // List available plugins
  async listPlugins(filters?: PluginFilters) {
    return this.db.plugins.find({
      category: filters?.category,
      type: filters?.type,
      verified: true,
      published: true
    }).sort({ downloads: -1 });
  }

  // Get plugin details
  async getPlugin(pluginId: string) {
    const plugin = await this.db.plugins.findById(pluginId);
    const reviews = await this.db.reviews.find({ pluginId });

    return {
      ...plugin,
      avgRating: this.calculateAvgRating(reviews),
      totalReviews: reviews.length
    };
  }

  // Install plugin
  async installPlugin(userId: string, pluginId: string) {
    const plugin = await this.getPlugin(pluginId);

    // Check if paid plugin
    if (plugin.price > 0) {
      const hasPurchased = await this.checkPurchase(userId, pluginId);
      if (!hasPurchased) {
        throw new Error('Plugin not purchased');
      }
    }

    // Download plugin package
    const packageData = await this.downloadPlugin(plugin);

    // Install to user's system
    await this.installToSystem(userId, packageData);

    // Record installation
    await this.db.installations.create({
      userId,
      pluginId,
      version: plugin.version,
      installedAt: new Date()
    });

    return { success: true };
  }

  // Submit new plugin (for developers)
  async submitPlugin(developerId: string, pluginData: PluginSubmission) {
    // Validate plugin package
    await this.validatePlugin(pluginData);
  }
}
```

```
// Security scan
const scanResult = await this.securityScan(pluginData);
if (!scanResult.passed) {
  throw new Error('Security scan failed');
}

// Create plugin entry
const plugin = await this.db.plugins.create({
  developerId,
  ...pluginData,
  status: 'pending_review',
  verified: false,
  createdAt: new Date()
});

// Notify review team
await this.notifyReviewTeam(plugin);

return plugin;
}
```

جديد Plugin كيفية إنشاء

الخطوات:



bash

1. إنشاء plugin من template
 npx @mnbara/create-plugin my-analytics-plugin

2. البنية الناتجة.

```
my-analytics-plugin/
├── src/
│   └── index.ts      # Main plugin file
├── tests/
│   └── index.test.ts # Tests
├── plugin.json      # Manifest
├── README.md
└── package.json
```

3. تطوير الـ plugin
 cd my-analytics-plugin
 npm install
 npm run dev

4. بناء الـ plugin
 npm run build

5. نشر على Marketplace
 npx @mnbara/publish-plugin

مثال بسيط: Google Analytics Plugin



typescript

```

// src/index.ts
import { MnbaraPlugin } from '@mnbara/plugin-sdk';

export default class GoogleAnalyticsPlugin extends MnbaraPlugin {
  private ga: any;

  getName() {
    return '@mnbara/google-analytics';
  }

  getVersion() {
    return '1.0.0';
  }

  async initialize() {
    // Initialize GA
    this.ga = await this.initializeGA(this.config.trackingId);

    // Hook into page views
    this.registerHook('page.view', (data) => {
      this.trackPageView(data);
    });

    // Hook into order completion
    this.registerHook('order.completed', (order) => {
      this.trackPurchase(order);
    });

    // Register admin UI
    this.registerAPI([
      {
        method: 'GET',
        path: '/analytics/dashboard',
        handler: this.getDashboardData.bind(this)
      }
    ]);
  }

  private trackPageView(data: any) {
    this.ga.pageview({
      page: data.path,
      title: data.title
    });
  }

  private trackPurchase(order: any) {
    this.ga.event('purchase', {
      transaction_id: order.id,
      value: order.total,
      currency: order.currency,
      items: order.items
    });
  }

  private async getDashboardData() {
    // Fetch analytics from GA API
    return {
      pageViews: await this.ga.getPageViews(),
      conversions: await this.ga.getConversions(),
    }
  }
}

```



```
    topProducts: await this.ga.getTopProducts()
  };
}

async destroy() {
  // Cleanup
}
}
```

Plugin Categories



typescript

// المتاحة Plugins معرفة أنواع الـ

```
export enum PluginCategory {
  // Payment
  PAYMENT_GATEWAY = 'payment-gateway',

  // Shipping
  SHIPPING_PROVIDER = 'shipping-provider',
  FULFILLMENT = 'fulfillment',

  // Marketing
  EMAIL_MARKETING = 'email-marketing',
  SMS_MARKETING = 'sms-marketing',
  PUSH_NOTIFICATIONS = 'push-notifications',

  // Analytics
  ANALYTICS = 'analytics',
  REPORTING = 'reporting',

  // Customer Service
  CHAT_SUPPORT = 'chat-support',
  TICKETING = 'ticketing',

  // Content
  CMS_INTEGRATION = 'cms-integration',
  MEDIA_MANAGEMENT = 'media-management',

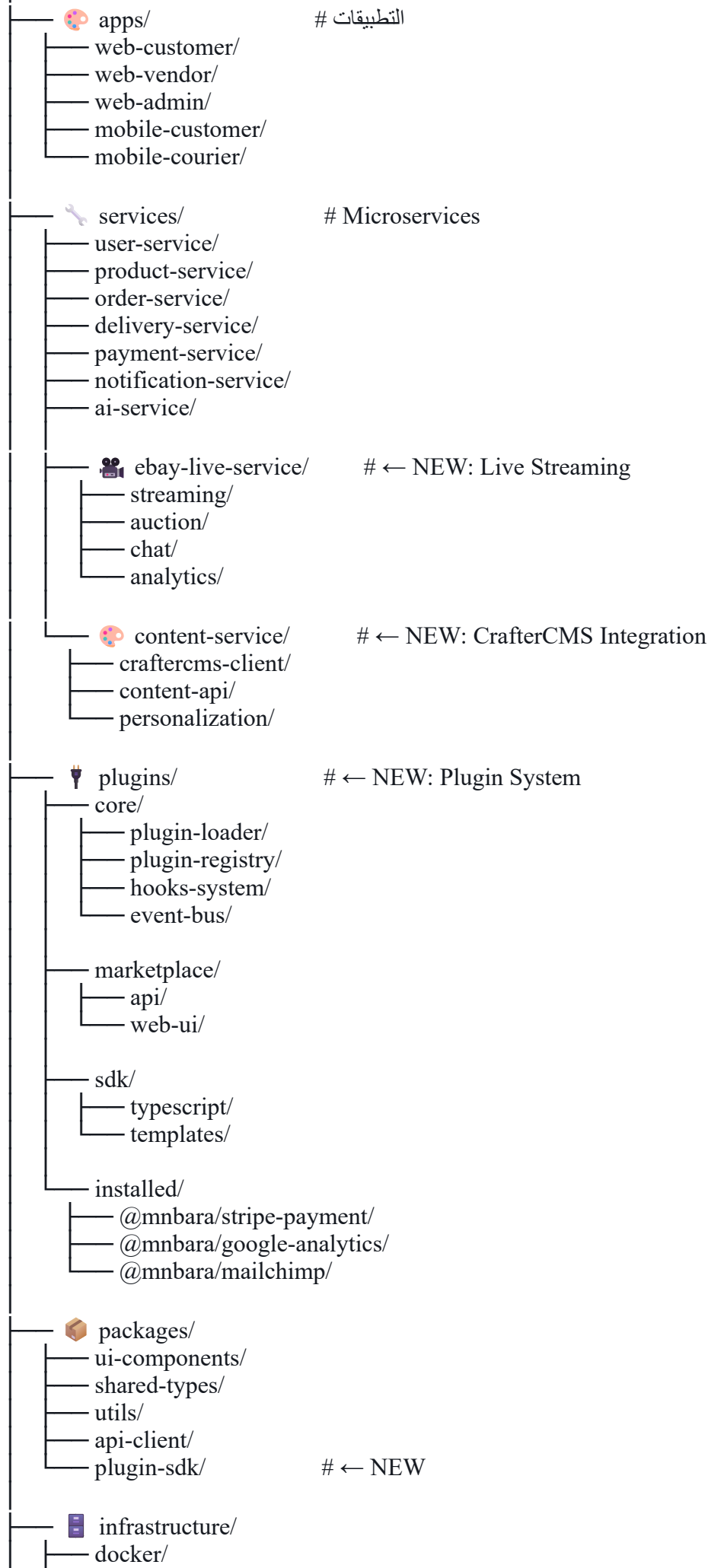
  // Social
  SOCIAL_COMMERCE = 'social-commerce',
  SOCIAL_LOGIN = 'social-login',

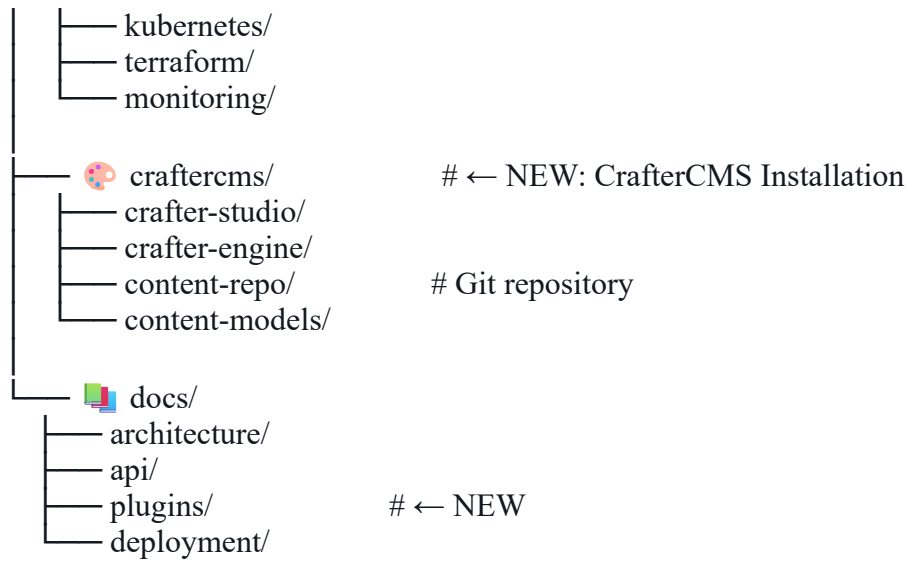
  // Marketplace
  LIVE_STREAMING = 'live-streaming',
  AUCTION = 'auction',

  // Utility
  SEO = 'seo',
  SECURITY = 'security',
  BACKUP = 'backup'
}
```

4 البنية الشاملة النهائية {#4-final-architecture}







5 خطة التنفيذ {#5-implementation-plan}

الإجمالي: 12-16 أسبوع Timeline



yaml

Phase 1: eBay Live Service (Weeks 1-6)

Week 1-2: MVP Streaming

- RTMP server setup
- Basic HLS conversion
- WebSocket chat
- Simple UI

Week 3-4: Auction System

- Bidding engine
- Soft-close mechanism
- Payment capture
- Product carousel

Week 5-6: Testing & Enhancement

- Load testing
- Latency optimization
- Analytics integration
- Documentation

Phase 2: CrafterCMS Integration (Weeks 7-10)

Week 7-8: Basic Setup

- CrafterCMS installation
- Content models creation
- API integration
- React components

Week 9-10: Advanced Features

- Personalization engine
- Multilingual support
- A/B testing setup
- AI content generation

Phase 3: Plugin System (Weeks 11-16)

Week 11-12: Core System

- Plugin loader
- Hook system
- Event bus
- SDK development

Week 13-14: Marketplace

- Marketplace API
- Web UI
- Payment integration
- Security scanning

Week 15-16: Official Plugins

- Payment gateways
- Analytics tools
- Marketing integrations
- Documentation & testing

Phase 4: Integration & Testing (Week 17-18)

- Complete system integration
- End-to-end testing
- Performance optimization
- Security audit
- Documentation finalization

الفريق المطلوب



yaml

eBay Live Team (3-4 developers):

- Backend Developer (Node.js, WebRTC)
- Frontend Developer (React, Video.js)
- DevOps Engineer (RTMP, Streaming)
- QA Engineer

CrafterCMS Team (2-3 developers):

- CrafterCMS Specialist
- Frontend Developer (React)
- Content Strategist

Plugin System Team (3-4 developers):

- Backend Architect
- SDK Developer
- Frontend Developer (Marketplace)
- Security Engineer

التكلفة التقديرية



yaml

Infrastructure (Monthly):

- Streaming Servers: \$500-1000
- CrafterCMS Hosting: \$200-500
- Database Scaling: \$300-600
- CDN for Content: \$200-400
- Total: ~\$1,200-2,500/month

Development (One-time):

- eBay Live: \$40,000-60,000
- CrafterCMS: \$25,000-35,000
- Plugin System: \$35,000-50,000
- Total: ~\$100,000-145,000

Maintenance (Monthly):

- Support & Updates: \$5,000-8,000

الخلاصة

إضافة كل ميزة:

الميزة	الوقت	التعقيد	الأولوية
eBay Live	عالي أسابيع 6	عالية	عالية
CrafterCMS	متوسط أسابيع 4	متوسطة	متوسطة
Plugin System	عالي جداً أسابيع 6	عالية جداً	عالية جداً

التوصية:

- 1. أولاً (أساس قابل للتوسع) Plugin System ابدأ بـ
- 2. رسمي plugin كأول eBay Live ثم
- 3. للمحتوى plugin كـ CrafterCMS ثم

الميزة الإضافية:



typescript
يمكنك إضافة أي خدمة لاحقاً بسهولة، Plugins بفضل نظام الـ //

// Example: TikTok Shop Plugin
npm @mnbara/create-plugin tiktok-shop
// 2-3 weeks development

// Example: Instagram Shopping Plugin
npm @mnbara/create-plugin instagram-shopping
// 2-3 weeks development

// Example: Amazon FBA Integration
npm @mnbara/create-plugin amazon-fba
// 3-4 weeks development

مع هذه البنية، منصتك ستكون جاهزة للتوسع في أي اتجاه 🚀